

Árbol Binario de Búsqueda para Organigrama Empresarial

Universidad: Universidad Técnica de Ambato

Carrera: Ingeniería de Software

Asignatura: Estructura de Datos (Tercer Nivel)

Estudiante: Paul Acurio

Fecha: 29/04/2026

1. Objetivo del Proyecto

Implementar en C++ una estructura de datos no lineal, específicamente un Árbol Binario de Búsqueda (BST), para registrar y organizar a los empleados de una empresa utilizando un código numérico como clave de jerarquía, permitiendo operaciones eficientes de inserción, búsqueda y lectura.

2. Explicación de la Estructura Utilizada

Para la representación del organigrama, se diseñó un **Árbol Binario de Búsqueda (BST)** desarrollado desde cero en C++. La elección de esta estructura sobre una lineal (como arreglos o listas enlazadas) se justifica por la naturaleza jerárquica de una empresa.

La implementación técnica se basó en:

- **Struct Empleado:** Actúa como el contenedor de la carga útil (payload), almacenando el código (clave), nombre y cargo de la persona.
- **Struct Nodo:** Es el bloque fundamental del árbol. Contiene los datos del empleado y dos punteros (izquierdo y derecho) que establecen las conexiones con los nodos subordinados.
- **Recursividad:** Los algoritmos de inserción, búsqueda, cálculo de altura y recorridos (inorden, preorden, postorden) se implementaron mediante funciones recursivas, lo que reduce la cantidad de código y aprovecha la pila de llamadas del sistema.

3. Definición de Conceptos Clave

Para comprender la topología del árbol implementado, se definen los siguientes elementos:

- **Raíz:** Es el nodo principal y punto de acceso a toda la estructura. Representa la cima del organigrama (Ej. La empresa misma o el CEO) y es el único nodo que no tiene un nodo superior.
- **Nodo Interno:** Cualquier nodo que tenga al menos un hijo subordinado (izquierdo o derecho). Representa a los cargos medios (Ej. Gerentes) que tienen personal a su cargo.
- **Hoja:** Nodos ubicados en los extremos inferiores de las ramas que no poseen ningún hijo (ambos punteros son nulos). Representan a los empleados base sin subordinados.
- **Nivel:** Indica la profundidad o la "generación" de un nodo. La raíz se encuentra en el nivel 0, sus hijos directos en el nivel 1, y así sucesivamente.
- **Altura:** Es la longitud del camino más largo desde el nodo raíz hasta la hoja más profunda. En nuestro sistema, indica la cantidad máxima de niveles jerárquicos en la empresa.

4. Análisis de Complejidad Temporal

El uso de un Árbol Binario de Búsqueda proporciona ventajas significativas en rendimiento:

- **Insertión y Búsqueda:** En un caso promedio, donde los códigos de los empleados se ingresan en un orden aleatorio. Esto significa que descartamos la mitad del árbol en cada paso de búsqueda. Sin embargo, en el peor de los casos (si se ingresan datos ordenados secuencialmente), el árbol se degenera en una lista enlazada, resultando en $O(n)$.
- **Recorridos (Inorden, Preorden, Postorden):** La complejidad es estrictamente de $O(n)$, ya que es obligatorio visitar cada uno de los nodos del organigrama exactamente una vez para imprimir la información.

5. Evidencia de Ejecución y Repositorio

El desarrollo colaborativo se gestionó mediante control de versiones. El código fuente, los *commits* individuales de cada integrante y la documentación base se encuentran alojados en el siguiente repositorio:

Enlace de GitHub: <https://github.com/Paul-Acurio/arbol-bst-empresa-cpp.git>

1. Menú principal y carga de datos:

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion:
```

2. Búsqueda de un empleado específico:

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 001

Empleado encontrado:
Codigo: 1 | Nombre: Roberto | Cargo: Bodegero

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 4

Recorrido Inorden:
Codigo: 1 | Nombre: Roberto | Cargo: Bodegero
```

3. Muestra de la Raíz:

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 3

Raiz del arbol:
Codigo: 1 | Nombre: Roberto | Cargo: Bodeguero
```

4. Recorridos (Inorden, Preorden, Postorden):

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 5

Recorrido Preorden:
Codigo: 1 | Nombre: Roberto | Cargo: Bodeguero
```

5. Cálculo de Altura y visualización de Nodos Hoja:

```

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 7

Altura del arbol: 1

===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raíz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 8

Nodos hoja:
Codigo: 1 | Nombre: Roberto | Cargo: Bodegero

```

6. Conclusiones

- La implementación del BST demostró ser una solución técnica altamente eficiente para modelar organigramas. Al aprovechar la propiedad del árbol (valores menores a la izquierda, mayores a la derecha), las búsquedas de empleados por código son exponencialmente más rápidas que revisar una lista completa.
- El uso de punteros dinámicos en C++ permitió optimizar el uso de memoria, asignando espacio en tiempo de ejecución únicamente cuando un nuevo empleado es contratado (insertado) en la empresa.
- La integración con Git y GitHub garantizó un flujo de trabajo ordenado, permitiendo a los distintos roles (programadores, líder y documentador) trabajar en paralelo sin pérdida de código ni conflictos en los archivos.